



Processes and Threads

Dept. of Computer Science

Hanyang University

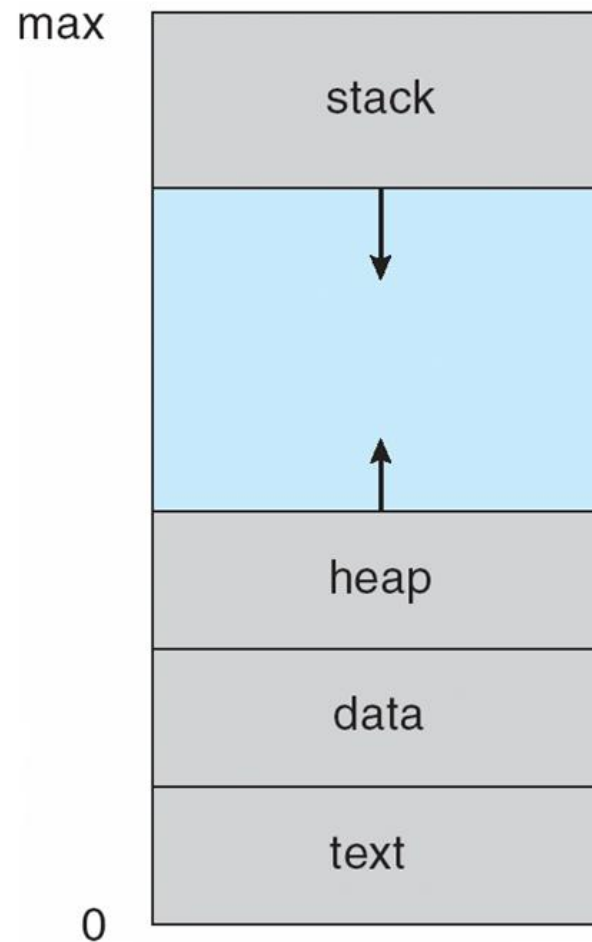




Process Concept

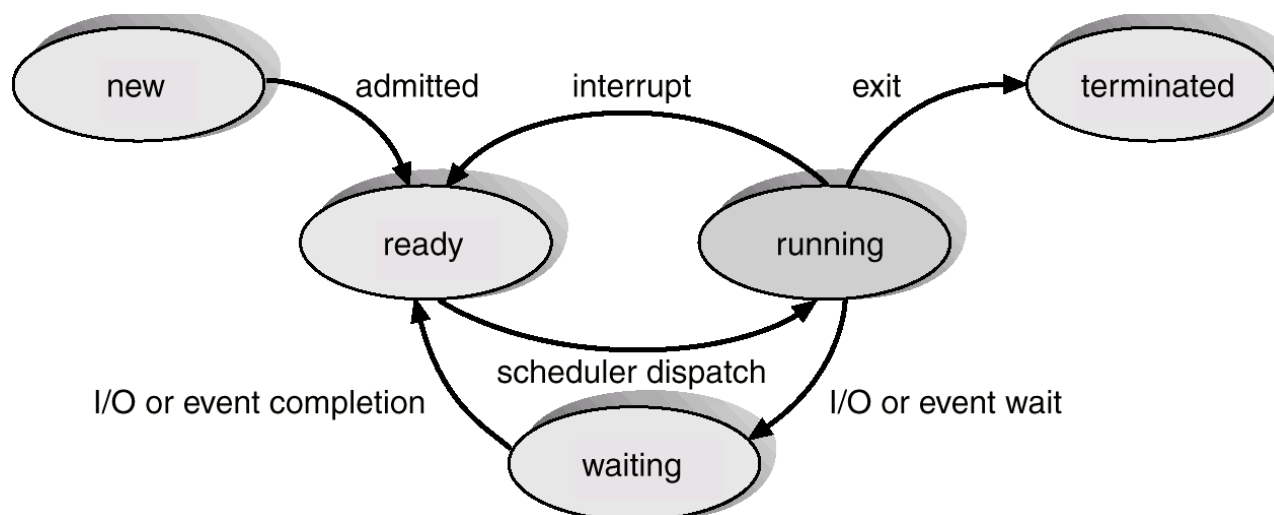
- Process
 - Program in execution
 - Program execution must progress in sequential fashion
- A process includes:
 - text section : program code
 - Stack : used in function calls and context switch
 - Function call: stores parameter, return address, local variables, etc
 - Context switch: stores register values including PC and stack pointer
 - data section : stores static and global variables
 - Heap section (free memory pool) for dynamic allocation

Process in Memory: Address Space



Process State

- As a process executes, it changes *state*
 - **new**: The process is being created
 - **running**: Instructions are being executed
 - **waiting**: The process is waiting for some event to occur
 - **ready**: The process is waiting to be assigned to a processor
 - **terminated**: The process has finished execution



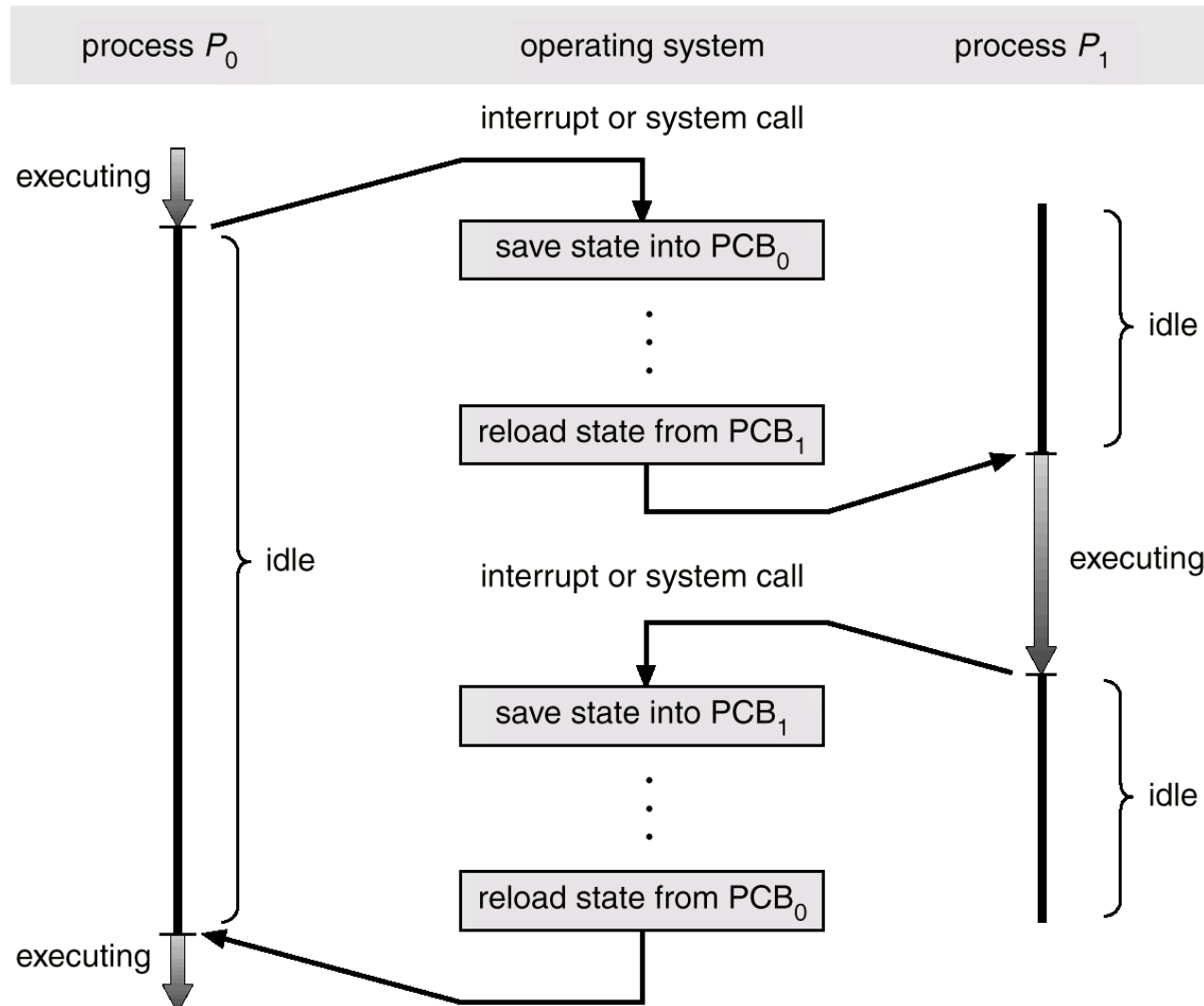
Basic State Transition Diagram of a Process



Process Control Block (PCB)

- Kernel **data structure** (eg. `task_struct` in Linux) representing a process that includes:
 - Process state
 - Program counter
 - CPU registers
 - CPU scheduling information
 - Memory-management information
 - Accounting information
 - I/O status information

CPU Switch from Process to Process

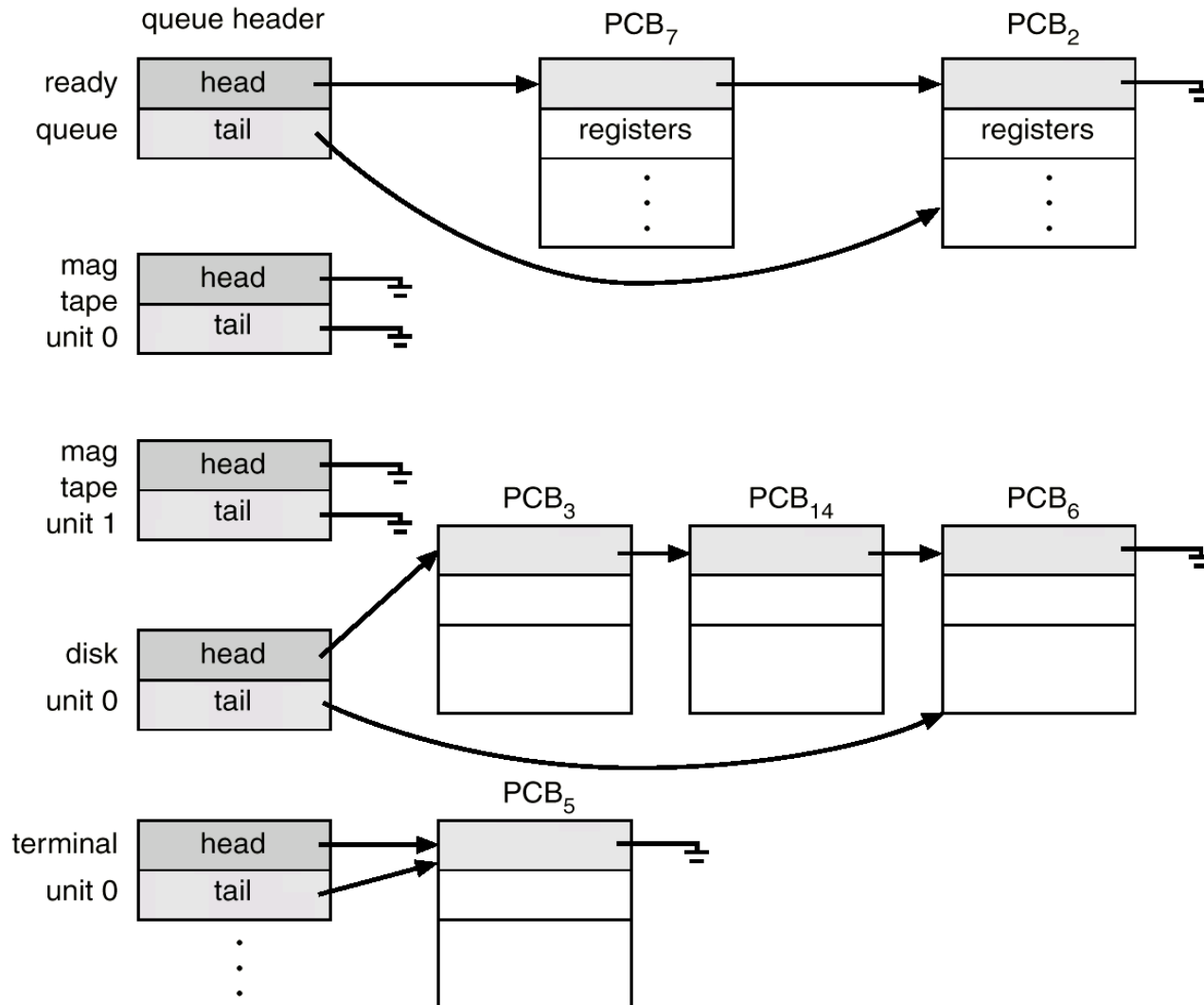




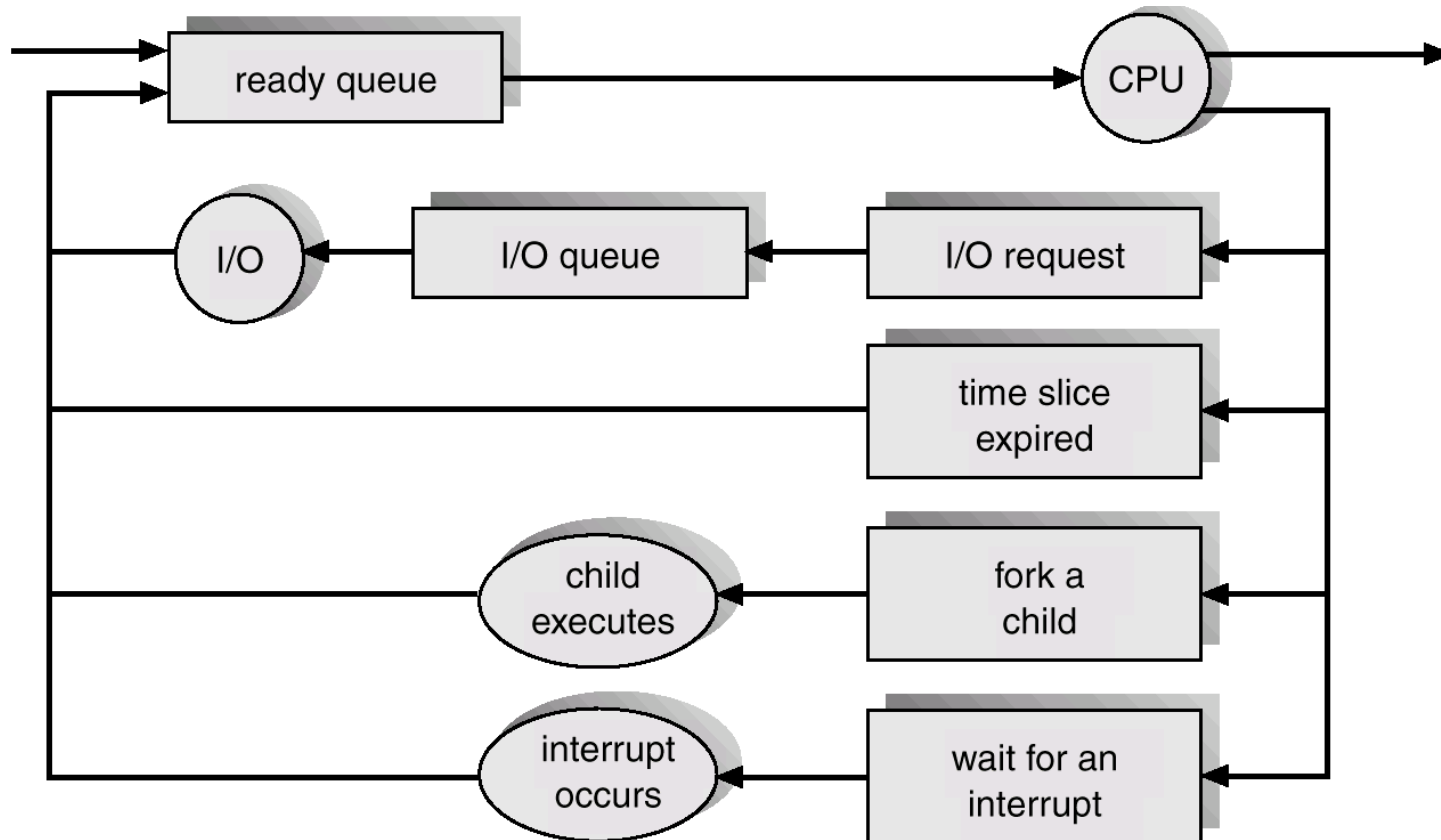
Process Scheduling Queues

- Job queue
 - set of all processes in the system
- Ready queue
 - set of all processes residing in main memory, ready and waiting to execute
- Device queues
 - set of processes waiting for an I/O device
- Process migration between the various queues

Ready Queue and Various I/O Device Queues

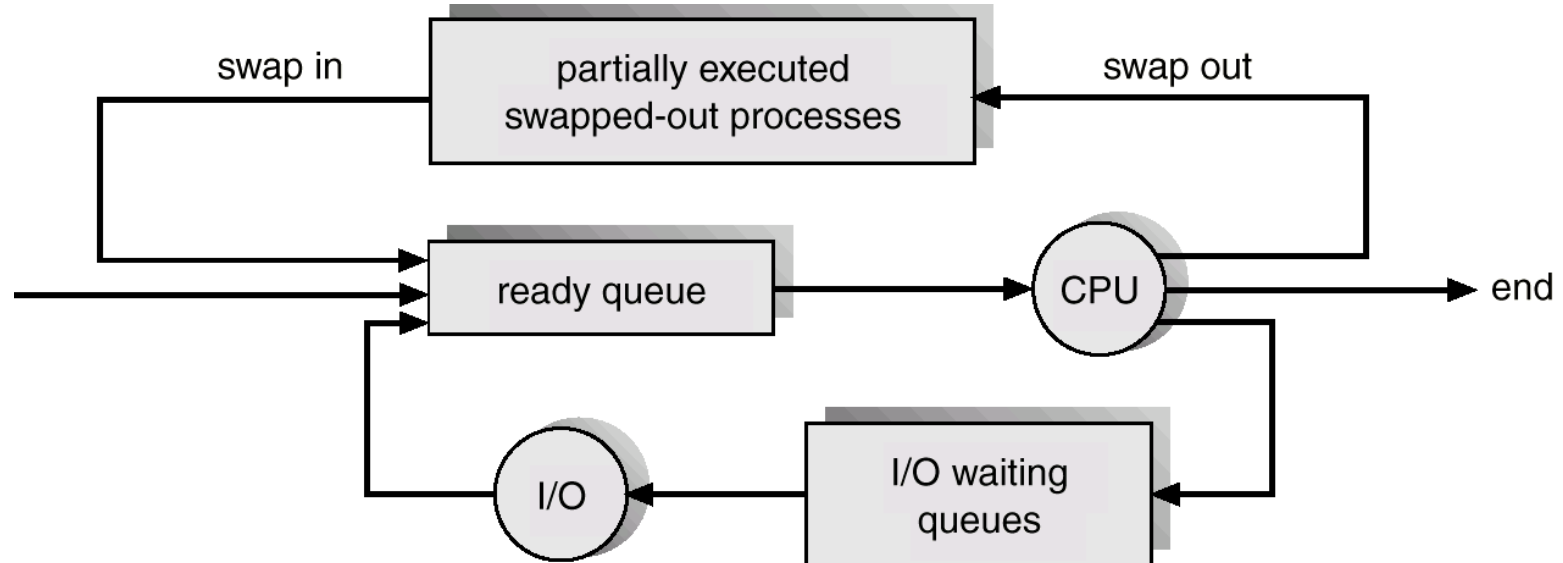


Representation of Process Scheduling



Schedulers

- Long-term scheduler (or job scheduler)
 - selects which processes should be brought into the ready queue
- Short-term scheduler (or CPU scheduler)
 - selects which process should be executed next and allocates CPU
- Addition of Medium-term scheduling





Schedulers (Cont.)

- Short-term scheduler is invoked very frequently (milliseconds) $\Rightarrow$ (must be fast)
- Long-term scheduler is invoked very infrequently (seconds, minutes) $\Rightarrow$ (may be slow)
- The long-term scheduler controls the *degree of multiprogramming*
- Processes can be described as either:
 - *I/O-bound process*
 - spends more time doing I/O than computations
 - many short CPU bursts
 - *CPU-bound process*
 - spends more time doing computations
 - few very long CPU bursts



Context Switch

- When CPU switches to another process
 - the system must save the state of the old process and
 - load the saved state for the new process via a context switch
- Context of a process represented in the PCB
 - Store: Registers → PCB
 - Restore: PCB → Registers
- Context-switch time is overhead
 - Mainly consists of memory operations: slow!
 - The system does no useful work while switching
 - Highly depends on hardware support
 - Intel CPU supports HW for context switching: state saving and restoring is done by HW, not by SW
 - Context switching accompanies cache flushing ➔ successive cache misses occur after context switch



Process Creation

- Parent process creates children processes, which, in turn create other processes, forming a tree of processes
- Processes are identified and managed via **process identifier (pid)**
- Resource sharing
 - Parent and children share all resources
 - Children share subset of parent's resources
 - Parent and child share no resources
- Execution
 - Parent and children execute concurrently
 - Parent waits until children terminate



Process Creation (Cont.)

- Address space
 - Child duplicate of parent
 - Child has a program loaded into it
- UNIX examples
 - `fork()` system call creates new process
 - `exec()` system call used after a `fork()` to replace the process' memory space with a new program



Process Creation (Cont.)

- **Starting point of the child process**
 - After `fork()` system call, the child process has exactly the same values of context (program counter, registers, memory info, open files info. etc) as the parent process
 - Hence, both the child process and the parent process executes from the same point:
 - after the `fork()` system call and before the return value arrives
 - The return value of `fork()` for child process is 0 while that for parent process is the PID of the child process

Process Creation (Cont.)

```
#include <stdio.h>
```

```
void main (int argc, char *argv[])
```

```
{
```

```
    int pid;
```

```
    /* fork another process */
```

```
    pid = fork();
```

```
    if (pid < 0) { /* error occurred */
```

```
        fprintf (stderr, "Fork Failed\n");
```

```
        exit(-1);
```

```
    }
```

```
    else if (pid == 0) { /* child process */  
        execlp("/bin/lis", "lis", NULL);
```

```
    }
```

```
    else { /* parent process */
```

```
        /* parent will wait for the child  
        to complete */
```

```
        wait(NULL);
```

```
        printf("Child Complete\n");
```

```
        exit(0);
```

```
    }
```

```
}
```



Process Termination

- Process executes last statement and asks the operating system to delete it (**exit**)
 - Output data from child to parent (via **wait**)
 - Process' resources are deallocated by operating system
- Parent may terminate execution of children processes (**abort**)
 - Child has exceeded allocated resources
 - Task assigned to child is no longer required
 - Parent is exiting and the operating system does not allow child to continue if its parent terminates
 - Cascading termination



Interprocess Communication

- Processes within a system may be independent or cooperating
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
 - Information sharing
 - Computation speedup
 - Modularity
 - Convenience
- Cooperating processes need **interprocess communication (IPC)**
- Two models of IPC
 - Shared memory
 - Message passing



Producer-Consumer Model

- Paradigm for cooperating processes with shared memory
 - *Producer* process produces data and stores it to the shared memory (buffer)
 - *Consumer* process consumes data in the shared memory
 - *unbounded-buffer* places no practical limit on the size of the buffer
 - *bounded-buffer* assumes that there is a fixed buffer size



Interprocess Communication – Message Passing

- Mechanism for processes to communicate and to synchronize their actions
- Message system – processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
 - **send**(*message*) – message size fixed or variable
 - **receive**(*message*)
- If P and Q wish to communicate, they need to:
 - establish a *communication link* between them
 - exchange messages via send/receive
- Implementation of communication link
 - physical (e.g., shared memory, hardware bus)
 - logical (e.g., logical properties)



Synchronization

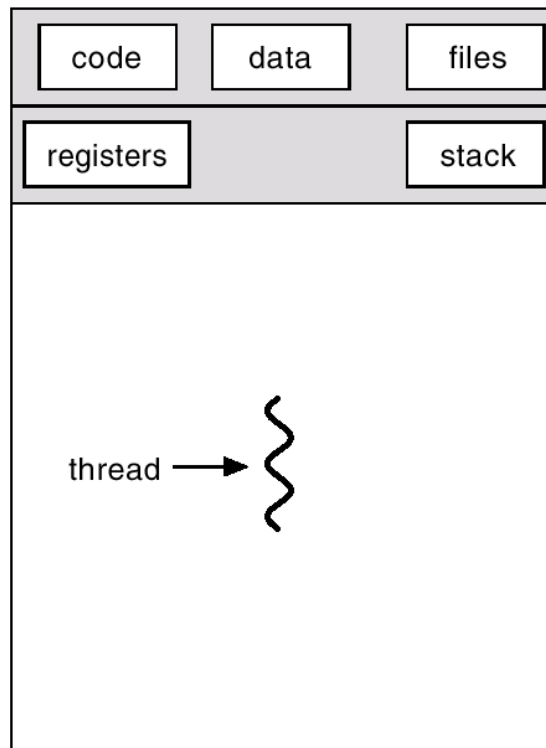
- Message passing may be either blocking or non-blocking
- **Blocking** is considered **synchronous**
 - **Blocking send** has the sender block until the message is received
 - **Blocking receive** has the receiver block until a message is available
- **Non-blocking** is considered **asynchronous**
 - **Non-blocking send** has the sender send the message and continue
 - **Non-blocking receive** has the receiver receive a valid message or null



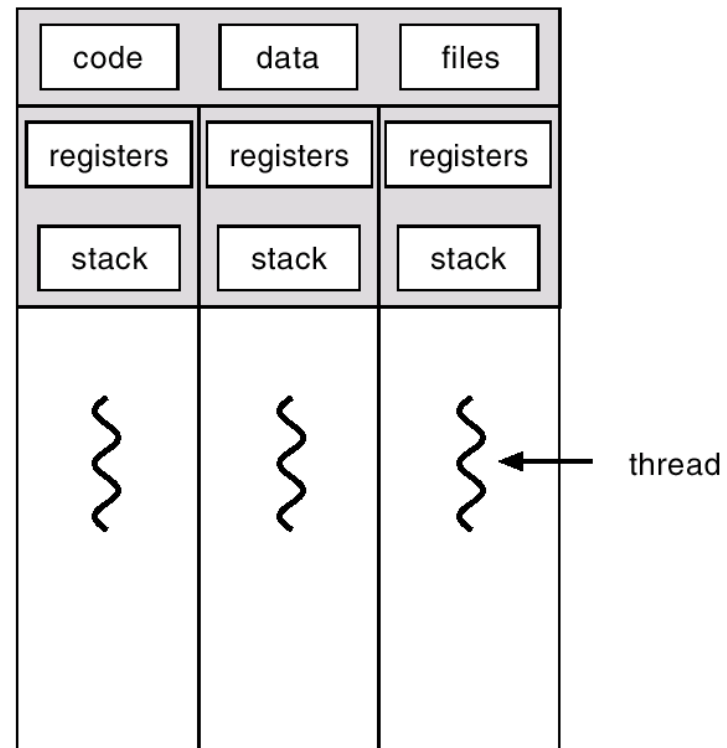
Chapter 5: Threads

- Overview
- Multithreading Models
- Threading Issues

Single and Multithreaded Processes



single-threaded



multithreaded



- A *thread* (or *lightweight process*) is a basic unit of CPU utilization; it consists of:
 - program counter
 - register set
 - stack space
- A thread shares with its peer threads its:
 - code section
 - data section
 - operating-system resources such as open files and signals
- A traditional or *heavyweight* process is equal to a task with one thread



- Responsiveness
 - Allow a program to continue running even if part of it is blocked or is performing a lengthy operation
- Resource Sharing
 - Several different threads of activity all within the same address space
- Economy
 - It is more economical to create and context switch threads than processes
- Scalability
 - Each thread may be running in parallel on a different processors



User Threads

- Thread management done by user-level threads library
 - Thread creation, scheduling and management
 - No support from the kernel
 - Fast to create and manage
 - With the single-threaded kernel, one user-level thread performing a blocking system call will cause the entire process to block
 - Since OS does not know the existence of other threads in that process, OS assigns CPU to other process
- Examples
 - POSIX *Pthreads*
 - Mach *C-threads*
 - Solaris *threads*



Kernel Threads

- Supported by the Kernel
 - Kernel performs creation, scheduling, management of threads : slower than user threads
 - A blocking thread does not block the entire process
 - In MP machine, the kernel can schedule threads on different processors
- Examples
 - Windows
 - Solaris
 - Tru64 UNIX
 - BeOS
 - Linux



Threading Issues

- Semantics of `fork()` and `exec()` system calls
 - How many threads should be created when one thread on a multithreaded program calls `fork()`?
 - Unix provides 2 versions of `fork()`'s depending on the application
 - `exec()` replaces the contents of the address space: all threads in the process are affected
 - If `exec()` is called immediately after forking, then duplicating all threads is not necessary
 - the number of threads should be determined by the new program
 - If `exec()` is not called after forking, all threads are duplicated
- Thread cancellation
 - Asynchronous cancellation
 - Immediately terminates the target thread
 - Resource de-allocation problem
 - Shared data integrity problem
 - Deferred cancellation
 - The target thread periodically check if it should terminate, allowing it an opportunity to terminate itself in an orderly fashion



Threading Issues (Cont'd)

- Signal Handling
 - Signals are used in UNIX systems to notify a process that a particular event has occurred
 - A **signal handler** is used to process signals
 1. Signal is generated by particular event
 2. Signal is delivered to a process
 3. Signal is handled
 - Options:
 - Deliver the signal to the thread to which the signal applies
 - Deliver the signal to every thread in the process
 - Deliver the signal to certain threads in the process
 - Assign a specific thread to receive all signals for the process



Threading Issues (Cont'd)

- Thread pools
 - Faster to service a request
 - Limits the number of threads that exist at any one point
- Thread specific data
 - Each thread might need its own copy of certain data
 - Ex. Transaction processing
 - We might service each transaction in a separate thread
 - Each thread should be assigned its unique identifier: the “tid” should be different each other, while all threads executes the same code, i.e., DB transaction processing code.



Multicore Programming

- Multicore systems putting pressure on programmers, challenges include
 - Dividing activities
 - Balance
 - Data splitting
 - Data dependency
 - Testing and debugging